

Explorability of Web Sources: Automated Resource Discovery via the wrx Framework

Cedric DECRUW ^{a,1}, Marc PORTIER ^a, Katrina EXTER ^a

^a*Flanders Marine Institute (VLIZ), Ostend, Belgium*

ORCID ID: Cedric Decruw <https://orcid.org/0000-0001-6387-5988>, Marc Portier

<https://orcid.org/0000-0002-9648-6484>, Katrina Exter

<https://orcid.org/0000-0002-5911-1536>

Abstract. Background/Motivation. Linked Open Data keeps growing, yet we still watch clients stall at the first hop because a human-facing URL rarely reveals where the RDF actually lives. Teams end up leaning on tribal knowledge about each publisher instead of following clues that should already be present on the web.

Objective. We present wrx (Web Resource Extraction), a TypeScript framework that tries to make discovery feel less like a scavenger hunt. It leans on the signposts already defined by the web so automated agents can move from a landing page to a machine-actionable description without bespoke rules.

Methods. wrx walks a cascading path: content negotiation, FAIR Signposting, RFC 9264 linksets, embedded RDF scripts, RFC 9727 / `.well-known/api-catalog`, and DCAT/sitemap fallbacks. Each step is scored and captured in a trace so the journey can be inspected or reproduced.

Results. Starting from a single URI, the framework resolves RDF across heterogeneous publisher setups and explains why a given representation was chosen through the recorded trace.

Conclusion. wrx turns discovery from ad hoc endpoint guessing into a transparent, protocol-defined process that leaves readers and machines with a clear narrative of how RDF was found.

Keywords. RDF discovery, FAIR Signposting, Linkset, API Catalog, content negotiation, Semantic Web, wrx

1. Introduction: The Discovery Barrier

The Semantic Web vision relies on interlinked, machine-readable data, yet we still spend far too much time coaxing RDF out of ordinary URLs. More often than not, the location of a machine-actionable description sits in project documentation, a wiki page, or behind a publisher-specific `/rdf` pattern that only insiders know about. Automation stalls as soon as that tribal knowledge is missing, leaving data technically published but effectively hidden from machines.

¹Corresponding Author: Cedric Decruw, cedric.decruw@vliz.be.

Discovery patterns are also scattered. One site uses HTTP Link headers, another expects content negotiation, and a third hides catalogue links inside a separate portal. Each technique is standardised, but clients rarely stitch them together. A bot that succeeds with content negotiation can still miss a richer description tucked away in a linkset or an API catalogue. Without a shared way to follow all of these signals, automation remains brittle.

We argue for making discovery boringly transparent: put the signposts on the wire so a client can follow them without special knowledge. wrx automates that idea. Given a single URI, it walks standard relations and catalogue structures to build a graph of resources and their descriptions, leaving behind a reproducible trace instead of a pile of guesswork. We also expect this client framework to guide service developers in exposing the necessary discovery hooks onto their published resources. Additionally this client availability can inspire browser plugins or native browser features into adding user-facing affordances onto discovered machine actionable content.

2. Background and Specifications

Explorability is grounded in several IETF and W3C standards that already exist on the web but are unevenly used. These standards define how resources can describe related representations, profiles, and catalogue entries. wrx treats them as a cascading set of hints that can be followed until an actionable RDF description is obtained.

2.1. RFC 8288: Web Linking

RFC 8288 defines the Web Linking model that underpins discovery on the web (Nottingham 2017). It standardises typed links between resources via HTTP Link headers and HTML `<link>` elements. Each link is a relation between a source and a context, described by a `rel` attribute. For example:

Link: `<https://example.org/metadata.ttl>; rel="describedby"; type="text/turtle"`

wrx uses this model throughout its cascade: it interprets relations such as `describedby`, `alternate`, `self`, `profile`, and `linkset` as explicit instructions for where to find metadata or alternative representations. RFC 8288 also allows additional link parameters (for example, `type`, `hreflang`, and `title`) that help wrx validate candidate resources and choose among competing links. The key strength of Web Linking is that it is content agnostic; a resource can advertise RDF, JSON, or human-oriented documentation using the same mechanism. This makes RFC 8288 the baseline protocol for uniform discovery.

Exactly this technique has been introduced in the field of scholarly publications as FAIR Signposting, a pragmatic profile of Web Linking to expose metadata without requiring a dedicated API (Van de Sompel, Klein, and Jones 2022).

2.2. RFC 9110 and RFC 2045: HTTP Semantics and Media Types

HTTP semantics define how representations are negotiated, cached, and validated (Fielding and Reschke 2022). For RDF discovery, status codes and content negotiation rules matter because they allow clients to request machine-readable formats without knowing

where they live. The Accept header and the Vary response header inform wrx when to attempt content negotiation or fall back to link relations. wrx respects redirects, content negotiation outcomes, and caching directives so that repeated queries can avoid redundant network traffic.

Media types (RFC 2045) provide the vocabulary for identifying RDF encodings (Freed and Borenstein 1996). Common RDF types such as `text/turtle`, `application/ld+json`, and `application/rdf+xml` are used as signals in both HTTP headers and link metadata. wrx validates these media types to avoid false positives, and prefers responses with explicit RDF content types to minimise ambiguity. Because some publishers use vendor-specific types, wrx also supports profile-driven recognition: a profile URI can indicate RDF semantics even if the media type is a custom JSON format.

2.3. RFC 6906 and RFC 7284: Profiles and Profile Registries

RFC 6906 defines the `profile` link relation, which differentiates the *concept* being represented from the bytes delivered on the wire (Wilde 2013). By linking to profile URIs, a publisher can state that a resource representation conforms to a specific conceptual model or shape. This is critical for automated clients that need to choose between multiple representations of the same resource, eg, a `schema.org` JSON-LD representation versus a DCAT catalogue entry. Profiles can be applied through Link headers, HTML link tags, or the `profile` parameter on Content-Type headers.

The Profile-URI registry (RFC 7284) standardises profile identifiers and provides discovery for well-known profiles (Wilde 2014). wrx uses profile URIs as first-class hints: if a link relation includes a known profile (eg, RO-Crate), the corresponding representation is ranked higher because it offers more predictable semantics. Profiles are also used to align HTTP-level discovery with RDF-level profile bridging (such as `dct:conformsTo` in DCAT catalogues), allowing wrx to unify conceptual semantics with actual representations.

2.4. RFC 9264: Linksets

RFC 9264 introduces linksets, a mechanism for grouping large collections of links in a separate document (Verborgh, Hochstenbach, and Van de Sompel 2022). Linksets solve the scalability problem of embedding numerous Link headers or HTML tags. A linkset is itself a first-class resource that can be described in JSON or plain-text form, and it uses the same relation types as RFC 8288. In wrx, linksets are treated as a high-confidence source of discovery. When a resource advertises a linkset via `rel="linkset"`, wrx dereferences the linkset, parses it, and incorporates its links into the discovery graph.

Linksets also support anchors and context targets, enabling a single linkset to describe relations for many resources. This makes them a natural fit for repositories, APIs, and dataset portals where a resource may have multiple representations or profile-specific descriptions. wrx uses linksets to locate RDF distributions, profiles, and alternative entry points without requiring publisher-specific URI templates.

2.5. RFC 8615 and RFC 9727: *.well-known* and API Catalog Discovery

The `/.well-known/` mechanism is standardised in RFC 8615 and provides a registered namespace for host-level discovery metadata (Nottingham 2019). RFC 9727 de-

defines `api-catalog` as both a Well-Known URI suffix and a link relation (Bergquist and Koch 2025). Concretely, publishers expose an API catalog via `/.well-known/api-catalog`, while also being able to advertise the same catalog from other resources using `rel="api-catalog"`.

RFC 9727 requires the catalog to be available as a Linkset (`application/linkset+json`, with the RFC 9727 profile URI), and allows additional representations through content negotiation. `wrx` uses this in two complementary ways: (1) it follows `api-catalog` links when they are present at resource level, and (2) it probes `/.well-known/api-catalog` as a host-level fallback. The resulting catalog links can then lead to API endpoints, machine-readable API descriptions (for example OpenAPI), and related metadata, which `wrx` incorporates into the broader RDF discovery graph.

2.6. RFC 9309 and Sitemaps: Host-Level Harvesting

RFC 9309 defines how crawlers retrieve and interpret `/robots.txt`, including the rule language (`user-agent`, `allow`, `disallow`) and handling behavior for redirects, unavailability, and caching (Koster, Iliev, and Zeller 2022). It also permits additional records such as Sitemap, which lets a publisher advertise one or more sitemap locations without coupling them to a specific `user-agent` group. `wrx` uses this as a host-bootstrap mechanism: when direct resource-level signals are missing, it resolves `/robots.txt`, extracts advertised sitemaps, and expands the candidate URI set from there.

Sitemaps then provide structured URL inventories plus optional hints such as `lastmod`, `changefreq`, and `priority` (Sitemaps.org 2024). However the base sitemap schema does not carry link nor profile semantics directly. In practice these have been introduced through the `xhtml:link` pattern (commonly used for alternate links) and can be reused to carry relation, profile hints per URL entry. This allows `wrx` to explore these link and profile semantics on a host-level.

This mechanism is used by `wrx` to recognise host-wide declared catalogues that in turn might contain additional semantics for the URI under exploration.

2.7. DCAT and Catalogue Bridging

The Data Catalog Vocabulary (DCAT) is a W3C standard for describing datasets and distributions (W3C 2020a). It is often used to publish dataset metadata in RDF, and its `dct:conformsTo` predicate can link datasets to formal profiles. `wrx` uses DCAT as the semantic refinement stage *after* host-level harvesting via RFC 9309 and sitemaps. In other words, `robots/sitemap` discovery identifies candidate catalogue resources; DCAT then bridges those catalogue entries to concrete data distributions. If a discovered catalogue entry advertises a distribution with an RDF media type, `wrx` can resolve that distribution as the actionable representation for the target resource.

DCAT provides explicit properties for `dcat:distribution`, `dcat:accessURL`, and `dcat:downloadURL`, which let `wrx` distinguish between an API endpoint and a static file distribution. Catalogues can also include `dcat:mediaType` or `dcat:format` declarations, which `wrx` uses to validate that a distribution is compatible with RDF. This makes DCAT the connective layer between host-level URL discovery (`robots/sitemaps`) and resource-level machine-actionable access.

2.8. JSON-LD Contexts

JSON-LD contexts allow JSON payloads to be interpreted as RDF by providing an external semantic mapping (W3C 2020b). This is critical for legacy APIs that emit JSON without embedded RDF semantics. wrx treats JSON-LD contexts as a conversion path: when a JSON response includes a `@context` or links to a context document, wrx can convert that payload into RDF and treat it as a valid representation. This extends explorability into ecosystems that are not explicitly RDF-native.

In practice, many APIs include lightweight JSON responses but publish a context file that maps keys to RDF vocabularies. wrx detects these contexts via HTTP headers, Link relations, or JSON-LD metadata embedded in the response. The conversion pipeline allows wrx to recover RDF semantics without requiring the publisher to expose separate RDF endpoints.

2.9. Additional Profile Registries and Community Standards

Beyond IETF and W3C standards, domain communities maintain profile registries and catalogue conventions. Examples include RO-Crate for research object packaging (RO-Crate Community 2024), Linked Data Event Streams (LDES) for streaming updates (LDES Community 2023), and scientific data conventions such as ERDDAP (NOAA ERDDAP 2024), OPeNDAP (OPeNDAP 2024), and CF Conventions (CF Conventions Community 2024). wrx treats these as profile families: if a linkset, API catalogue, or profile URI points to one of these registries, wrx can prioritise the corresponding metadata or endpoint because it aligns with established community semantics.

For example, RO-Crate defines a canonical `ro-crate-metadata.json` entry that describes a research object and its linked resources in a predictable, machine-actionable form. wrx can prioritise this entry when profile hints point to RO-Crate and then follow its linked entities to recover richer metadata context. LDES resources likewise describe streams that can be consumed incrementally. These profiles expand the reach of explorability into domains that already maintain discovery infrastructure, even if they are not explicitly RDF-first.

3. Concept: Explorability vs. Findability

Findability creates a chicken-and-egg problem: a resource is findable only if a client already knows where a publisher placed it, or if it is registered in a specific catalogue. Explorability avoids this by placing a navigation map within or alongside the resource itself. In this analogy:

- The URI is the You are here marker.
- `rel="profile"` functions as the legend explaining the conceptual type of the resource.
- Linksets define nearby routes and alternative representations.
- Sitemaps and robots.txt act as the region map.
- API catalogues describe the road network for programmatic interfaces.

Explorability focuses on *conceptual resources* rather than only byte-level representations. HTTP semantics distinguish between a resource and its representations (Fielding and Reschke 2022). wrx therefore treats a landing page, a JSON-LD representation, and an RDF distribution as different manifestations of a common concept. By traversing links and catalogues, wrx constructs a graph that maps these manifestations and provides a reproducible trail of how each representation was found.

Table 1 summarises how core standards and mechanisms contribute to the exploration strategy.

Standard / mechanism	Discovery role in wrx	Typical signal
RFC 8288	Base linking model for discovery	Link headers, HTML <code><link></code> tags
HTML + JSON-LD embedding	In-document RDF extraction	<code><script type="application/ld+json"></code> blocks
RFC 6906	Conceptual profile identification	<code>rel="profile"</code>
RFC 9264	External link collections	<code>rel="linkset" + linkset</code> document
RFC 9727	Host-level API discovery	<code>/.well-known/api-catalog</code>
RFC 9309 + Sitemaps	Domain-level harvesting	<code>robots.txt</code> , <code>sitemap</code> indexes
DCAT	Catalogue to distribution bridge	<code>dcat:distribution</code> with RDF media type
JSON-LD	Semantify JSON representations	<code>@context</code> or <code>context</code> link

Explorability also reframes interoperability: instead of requiring every client to implement bespoke discovery rules, publishers can expose standardised signposts that any conforming agent can follow. This reduces coupling and allows data producers to evolve their infrastructure without breaking downstream consumers.

4. The wrx Discovery Framework

wrx is a TypeScript module targeting the Bun runtime. It implements a cascading discovery algorithm that starts with the most precise signals and gracefully relaxes to more generic strategies. The client supplies a URI, and wrx returns the best available RDF representation plus a trace of the steps taken.

4.1. Design Goals

The library is guided by four design goals:

1. **Protocol-first discovery:** only follow standardised signals, avoiding publisher-specific heuristics.
2. **Graceful degradation:** return the first acceptable RDF representation but allow deeper exploration when needed.
3. **Traceability:** provide a structured log of discovery steps and their confidence levels.
4. **Interoperability:** support multiple RDF serialisations and profiles with minimal configuration.

4.2. Discovery Cascade

The current implementation supports the following strategies, ordered from most specific to most general:

1. **Direct content negotiation:** issue HTTP requests with RDF Accept headers and capture negotiated representations. This leverages RFC 9110 semantics and RFC 2045 media types to detect RDF on the first request.
2. **FAIR Signposting:** inspect HTTP Link headers for `describedby`, `type`, `self`, and `linkset` relations to locate metadata objects.
3. **HTML link parsing:** scan HTML for `rel="alternate"`, `rel="describedby"`, and `rel="linkset"` declarations that point to RDF or linkset documents.
4. **Linkset resolution:** fetch and parse RFC 9264 linkset documents to find additional representations or profiles. Each linkset is validated for supported media types and profile relations.
5. **Embedded RDF scripts:** extract RDF from `<script type="application/ld+json">` blocks and other embedded data scripts when present, converting JSON-LD to RDF when necessary.
6. **API catalogue discovery:** resolve `/.well-known/api-catalog` endpoints as described in RFC 9727 to enumerate API entry points and schemas.
7. **Robots.txt + sitemap harvesting:** parse `robots.txt` (RFC 9309) and sitemaps to discover catalogue pages or dataset distributions and link semantic hints.
8. **Catalogue bridging:** follow DCAT catalogues to locate RDF distributions and cross-reference `dct:conformsTo` profile assertions.

Each step includes validation of media types, profiles, and dereferenceability to avoid false positives. The cascade is intentionally conservative: any step that yields an authoritative RDF representation can terminate the process, while subsequent steps are used to enrich the discovery graph.

4.3. Indirect, Direct, and Host Harvesting

wrx groups discovery into three modes that correspond to increasing scope:

- **Indirect harvesting:** link relations and linksets discovered in headers or HTML, where the publisher explicitly describes the resource.
- **Direct harvesting:** content negotiation and embedded scripts, where the representation itself contains RDF or RDF-compatible data.
- **Host harvesting:** catalogues, sitemaps, and well-known endpoints that expose domain-wide metadata.

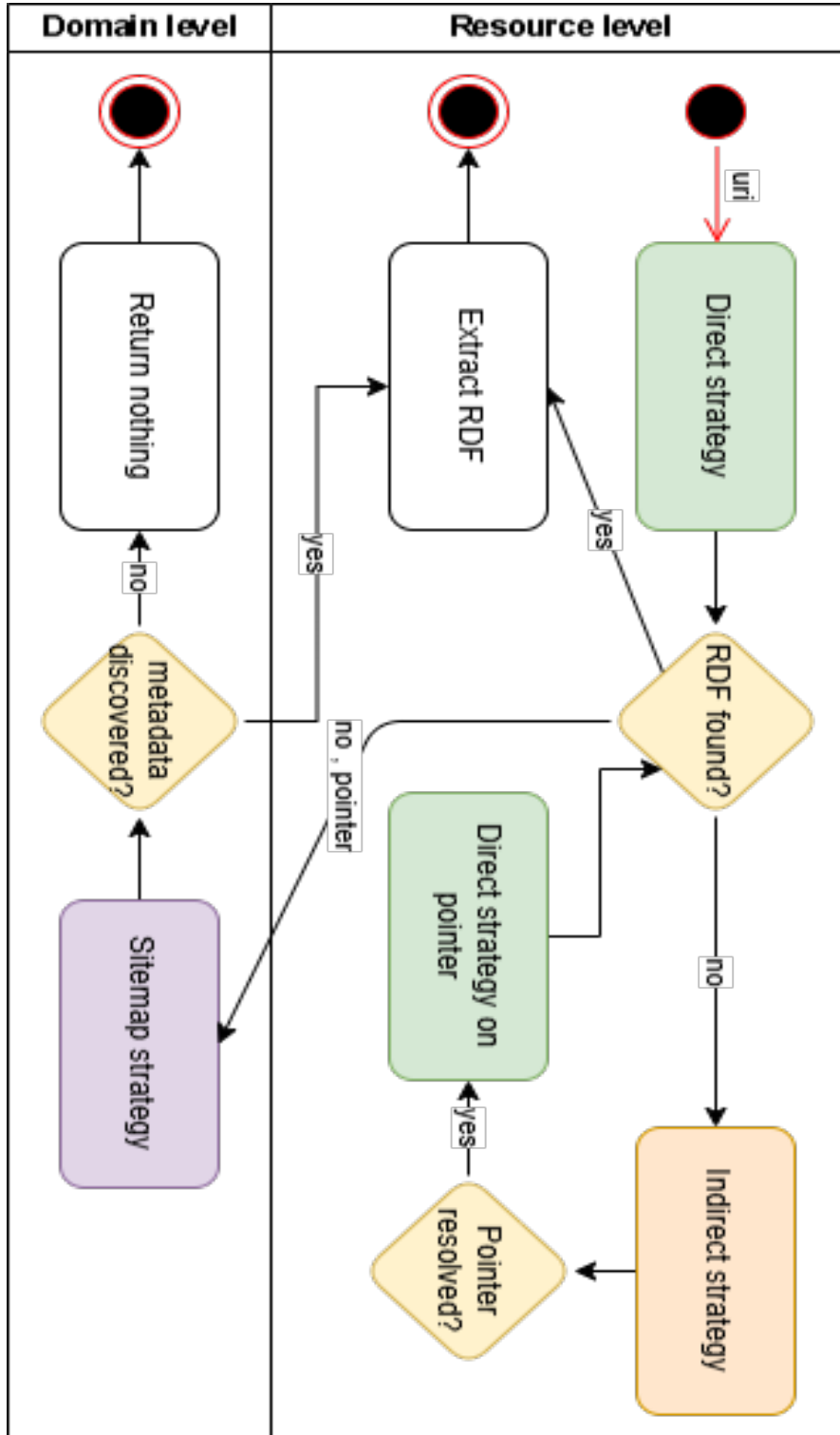


Figure 1. Strategic overview of the wrx discovery framework and how cascading discovery modes relate to one another.

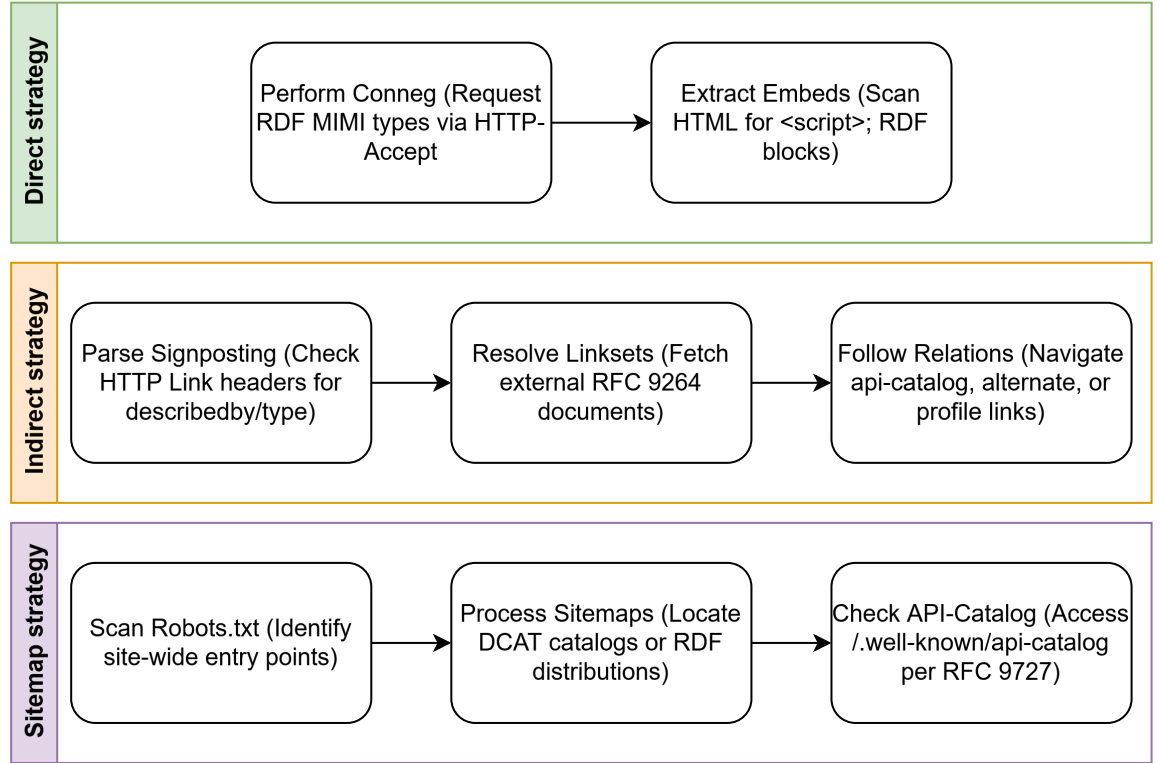


Figure 2. Detailed strategy breakdown used by wrx to evaluate and combine discovery signals.

This stratification allows clients to choose a latency and completeness tradeoff by configuring how far the cascade should proceed. For example, a UI that needs fast rendering might stop after indirect or direct harvesting, while a batch harvester can proceed into host-level discovery for maximum completeness.

4.4. Scoring and Trace Model

Each discovery step generates a trace entry capturing: (1) the source URL; (2) relation types followed; (3) the media type and profile; (4) a confidence score; and (5) any conversion steps applied. Scores are higher when the signal is explicit (for example, `rel="describedby"` with an RDF media type) and lower when inferred (for example, a sitemap entry lacking format metadata). The trace model is essential for observability: it allows downstream systems to explain how a representation was found and to select the most reliable version when multiple options are available.

4.5. Conversion to RDF

Not all discoverable representations are RDF-native. JSON-LD contexts, API catalogue entries in JSON, and HTML-embedded microdata can all be converted into RDF. wrx provides conversion hooks that interpret JSON-LD payloads or fetch context documents to transform structured JSON into RDF graphs. This allows wrx not only to locate RDF data but also to *derive* RDF from semantically annotated representations.

4.6. Example Walkthrough

Consider a DOI landing page that returns HTML. wrx first attempts content negotiation for RDF types; if that fails, it inspects HTTP Link headers for `describedby` relations. Suppose a `describedby` link points to a JSON record with an accompanying JSON-LD context. wrx resolves the link, detects the `@context`, converts the JSON to RDF, and returns the RDF graph along with a trace noting the signposting link and the conversion step. If no `describedby` link exists, wrx falls back to parsing HTML link elements, then to a linkset, and finally to host-level catalogues.

This walkthrough illustrates how wrx can discover RDF even when the initial resource lacks explicit RDF representations, by chaining standardised hints and semantic conversions.

5. Implementation Details

The wrx library is implemented in TypeScript and targets the Bun runtime for modern web-compatible HTTP APIs and high-performance parsing. Key components include:

- **HTTP client layer:** executes requests with configurable headers, retries, and redirect handling consistent with RFC 9110 semantics.
- **Link parser:** normalises RFC 8288 links from HTTP headers and HTML `<link>` tags into a unified internal representation.
- **Linkset parser:** reads RFC 9264 linkset documents in JSON or text format and extracts typed link relations.
- **Conversion module:** handles JSON-LD expansion and RDF serialisation selection based on requested or detected media types.

Implementation choices prioritise deterministic behaviour. For example, wrx avoids crawling arbitrary HTML links unless a defined relation is present. It also limits cross-domain traversal to avoid accidental drift into unrelated resources, unless the publisher explicitly provides cross-domain relations.

5.1. Data Model

wrx models discovery as three connected layers rather than a single flat graph. First, it keeps a **resource graph** in which nodes are conceptual resources and edges are typed relations (`describedby`, `alternate`, `profile`, `linkset`, `api-catalog`, etc.). Second, it keeps an **observation graph** that records where each relation was seen (HTTP Link header, HTML `<link>`, linkset record, sitemap entry, or embedded JSON-LD script), so relation claims are traceable to concrete web evidence. Third, it keeps a **process trace** with step mode (`indirect`, `direct`, `host`), source URI, dereference outcome, normalisation decisions, and conversion events.

This structure follows the exploration model used in the design material: `indirect` mode prioritises link-based navigation (headers, embedded links, linksets, redirects), `direct` mode prioritises immediate representation retrieval (content negotiation, redirects, embedded RDF), and `host` mode bootstraps from `/robots.txt` to sitemaps and catalogues. Linksets and sitemap hints are converted into RDF statements and attached to the same graph with provenance-style annotations, so clients can query either the resulting resource relations or the evidence chain that produced them.

5.2. Normalisation

Discovery evidence often arrives in different URI forms across headers, embedded HTML links, linksets, sitemaps, and catalogues. wrx applies a normalisation step before graph insertion: it resolves relative references against their context URI, follows redirects to a stable absolute URI, and applies syntax normalisation while preserving semantics (including meaningful fragments). Source location, discovery mode, and retrieval metadata remain attached as provenance.

Example: a page at `https://example.org/resource/42` exposes `<link rel="describedby" href="../../meta/42">`, while the server redirects that target to `https://example.org/meta/42.jsonld`. wrx resolves the relative link against the page context, follows the redirect, and stores the normalised target as `https://example.org/resource/42` in the discovery graph, with trace metadata recording the original relative href and redirect chain.

This example plays out slightly different if any of the visited resources exposed a `<link rel="self" href="/id/42">`. In this case the normalised target will be `https://example.org/id/42`.

5.3. Extensibility

The cascade is implemented as a pipeline of strategy modules. Each module receives the current discovery graph and can append new nodes and edges. This allows deployments to insert custom strategies (for example, a domain-specific catalogue format) without rewriting the core. The scoring model can also be configured to favor certain profiles or media types.

6. Evaluation and Use Cases

Because wrx is a discovery library rather than a single application, evaluation focuses on practical discovery outcomes. Pilot experiments against FAIR-compliant repositories show that a single `describedby` link often resolves directly to an RDF metadata record. In less curated environments, wrx typically succeeds by following HTML alternates to linksets and then to profile descriptions.

Representative use cases include:

- **Pre-flight discovery for digital assistants:** before invoking an API, wrx extracts the API catalogue and profiles to select the correct endpoint and representation.
- **Metadata completion for research objects:** given only a DOI landing page, wrx follows signposts to authoritative RDF descriptions suitable for repository ingest and curation.
- **Zero-configuration UI composition:** paired with RDF Web Components, wrx can feed a client-side renderer that turns arbitrary URIs into templated views.
- **Cross-domain dataset federation:** by following DCAT catalogues and profile links, wrx can locate RDF distributions across multiple organisations without prior agreements on endpoint naming.
- **Containerised standards-conformant LOD testbed:** a reproducible LOD server can be hosted via Docker (https://github.com/vliz-be-opsci/lod_docker_webserver), exposing resources that implement the papers discovery standards (for example

Web Linking, linksets, profiles, API catalogues, and sitemap-based host discovery). This provides a practical environment for wrx validation, interoperability testing, and demonstration.

Even when discovery fails, the trace reveals which signals were missing, which can inform publishers on how to improve explorability.

7. Related Work and Positioning

Several systems have addressed RDF discovery through either content negotiation or specialised endpoints. Linked Data browsers and SPARQL services provide rich interfaces but require prior knowledge of where the data lives. Metadata harvesters rely on catalogues but are limited to domains that publish them. wrx positions itself between these extremes by combining lightweight protocol signals (link headers, profiles, and linksets) with catalogue-level discovery. This hybrid approach supports both immediate resource-level discovery and broader host-level exploration.

8. Discussion: Adoption and Limitations

Explorability depends on publishers emitting meaningful links. wrx cannot conjure metadata that is absent, and it inherits the availability and correctness of remote resources. Adoption challenges include incomplete use of link relations, inconsistent media type declarations, and uneven operational quality of discovery endpoints (stale linksets, incomplete API catalogues, or missing robots/sitemap maintenance). In practice, discovery failures are often not protocol failures but publication-governance failures: relations are present but underspecified, outdated, or disconnected from the actual resource lifecycle.

Another structural limitation is the insufficient cross-domain adoption and curation of profile URI registries, despite the existence of registry mechanisms. This is a major issue for linking semantics on the web: without stable and discoverable profile identifiers, clients cannot reliably interpret what a declared `rel="profile"` actually means, compare equivalence across publishers, or rank competing representations with confidence. The result is semantic fragmentation, where links remain technically valid but operationally ambiguous.

RDF conversion from JSON or HTML also relies on consistent context publication; for JSON-LD-derived interpretation paths, missing or invalid contexts prevent unambiguous semantic interpretation. Likewise, host-level harvesting can expose candidate resources but cannot by itself guarantee conceptual alignment unless profile declarations and catalogue metadata are maintained with the same rigor as the primary data endpoints.

Nevertheless, the framework demonstrates that existing standards are sufficient for discovery when they are used consistently. wrx encourages small, incremental publisher changes (eg, adding a Link header or a `.well-known/api-catalog`) that can improve machine-accessible transparency quickly. The trace model also provides actionable feedback to publishers by highlighting which signposts are missing or inconsistent.

9. Future Work

Future work focuses on richer profile negotiation, and optional caching layers to reduce redundant exploration of large sites. Another priority is expanding support for community profile registries, allowing wrx to better serve domain-specific data ecosystems. The trace model could also be extended with provenance metadata so that discovery paths can be cited or audited in scholarly contexts.

10. Conclusion

Explorability reframes discovery as an on-the-fly, standards-driven process rather than a catalogue lookup. wrx demonstrates that existing web standards are sufficient to automate this journey today. By cascading through signposts, linksets, catalogues, and embedded scripts, wrx turns a URL into a machine-actionable graph with minimal prior knowledge, moving the Semantic Web closer to everyday automation.

11. Declaration on Generative AI

This manuscript was prepared with support from generative AI tooling for language refinement and structural editing. The system description was defined by the authors.

12. Ethical Considerations and Environmental Footprint

Ethical considerations. wrx processes publicly accessible web resources and follows explicit protocol-level discovery signals exposed by publishers. It does not bypass access controls, and it inherits usage constraints and licensing terms from the discovered resources and endpoints.

Environmental footprint. The framework is designed for lightweight HTTP-based discovery with bounded cascading steps and optional caching, reducing redundant requests and computational overhead during repeated discovery workflows.

References

- Bergquist, Tobias, and Andreas Koch. 2025. Api-Catalog: A Well-Known URI and Link Relation to Help Discovery of APIs. <https://www.rfc-editor.org/rfc/rfc9727>.
- CF Conventions Community. 2024. Climate and Forecast (CF) Metadata Conventions. <https://cfconventions.org/>.
- Fielding, Roy, and Julian Reschke. 2022. HTTP Semantics. <https://www.rfc-editor.org/rfc/rfc9110>.
- Freed, Ned, and Nathaniel Borenstein. 1996. Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies. <https://www.rfc-editor.org/rfc/rfc2045>.

- Koster, Martijn, Dimitar Iliev, and Lene Zeller. 2022. Robots Exclusion Protocol. <https://www.rfc-editor.org/rfc/rfc9309>.
- LDES Community. 2023. Linked Data Event Streams. <https://semiceu.github.io/LinkedDataEventStreams/>.
- NOAA ERDDAP. 2024. ERDDAP: Environmental Research Divisions Data Access Program. <https://coastwatch.pfeg.noaa.gov/erddap/>.
- Nottingham, Mark. 2017. Web Linking. <https://www.rfc-editor.org/rfc/rfc8288>.
- . 2019. Well-Known Uniform Resource Identifiers (URIs). <https://www.rfc-editor.org/rfc/rfc8615>.
- OPeNDAP. 2024. OPeNDAP: Open-Source Project for a Network Data Access Protocol. <https://www.opendap.org/>.
- RO-Crate Community. 2024. RO-Crate Metadata Specification. <https://www.researchobject.org/ro-crate/>.
- Sitemaps.org. 2024. Sitemaps XML Format. <https://www.sitemaps.org/protocol.html>.
- Van de Sompel, Herbert, Martin Klein, and Shawn Jones. 2022. FAIR Signposting Profile. <https://signposting.org/FAIR/>.
- Verborgh, Ruben, Patrick Hochstenbach, and Herbert Van de Sompel. 2022. Linkset: Media Type and Link Relation Type. <https://www.rfc-editor.org/rfc/rfc9264>.
- W3C. 2020a. Data Catalog Vocabulary (DCAT) - Version 2. <https://www.w3.org/TR/vocab-dcat-2/>.
- . 2020b. JSON-LD 1.1. <https://www.w3.org/TR/json-ld11/>.
- Wilde, Erik. 2013. The "Profile" Link Relation Type. <https://www.rfc-editor.org/rfc/rfc6906>.
- . 2014. The Profile URI Registry. <https://www.rfc-editor.org/rfc/rfc7284>.